Lab 3 — SQL kontra Apache Solr

Wyszukiwanie pełnotekstowe, facety i pomiary skalowania

Autor: Michal _____ **Data:** _____

1. Cel i zakres

Celem projektu jest porównanie PostgreSQL z Apache Solr podczas wyszukiwania w dużym zbiorze artykułów. Wykonano import danych, indeksowanie pełnotekstowe, przykładowe zapytania, facety oraz testy skalowania dla 50 000, 250 000 i 500 000 rekordów.

Systemy uruchomiono w Dockerze. Wykorzystano PostgreSQL 17, Apache Solr 10 i Python 3.12.3.

Środowisko pomiarowe: Ubuntu 24.04.5 LTS, Intel Core i5-11400H (6 rdzeni / 12 wątków), 32 GB RAM, dysk Samsung NVMe; Solr z `SOLR_HEAP=2g`.

2. Dane

Zbiór pochodzi z Hugging Face: wikimedia/structured-wikipedia, konfiguracja `enwiki_namespace_0` (angielska Wikipedia). Karta zbioru deklaruje licencję CC BY-SA 4.0; artykuły źródłowe Wikimedia mogą mieć dodatkowe warunki atrybucji, które należy zachować przy redystrybucji. Plik `data-set/wikipedia.csv` ma 500 000 rekordów i około 843 MiB.

Kontrola	Wynik
Rekordy / unikalne ID	500 000 / 500 000
Puste wartości	0 w kontrolowanych kolumnach
Długość tekstu	średnio 1 686 znaków; minimum 300
Kategorie	`topic` — 7 wartości; `article_type` — 6 wartości
Data	500 000 poprawnych wartości `date_modified`

Przykładowy rekord: ID `72535323`, tytuł `"Khan gizi" spring`, kategoria `geography`; tekst zaczyna się od: „Khan gizi spring is a spring located next to Khan's daughter Natevan's palace in the Çöl Qala neighborhood of Shusha.”

Tekst w loaderze danych jest ograniczony do maksymalnie 5000 znaków. Kategorie są tworzone heurystycznie w `data-set/download.py`, a nie pochodzą bezpośrednio z etykiet eksperckich.

3. PostgreSQL

Tabela `wikipedia_articles` zawiera `id BIGINT PRIMARY KEY`, tytuł, tekst, kategorie, datę oraz `search_vector TSVECTOR`. Wektor wyszukiwania budowany jest z tytułu i tekstu za pomocą angielskiej konfiguracji PostgreSQL. Indeks `GIN` przyspiesza operator `@@`.

Zapytania obejmują wyszukiwanie słowa `run`, porównanie z `LIKE '%run%'`, frazę `United States` oraz warunek logiczny `history AND (war OR peace) AND NOT fiction`.

Zapytanie SQL	Dopasowane artykuły	Przykładowy czas
Full text: `run`	28 800	75,036 ms
`LIKE` : `%run%`	48 804	782,404 ms
Fraza: `United States`	38 148	222,591 ms
Warunek AND/OR/NOT	4 240	13,238 ms

To pojedyncze pomiary z pierwszego testu, a nie mediany pięciu powtórzeń.

4. Apache Solr

W core `wikipedia` zaindeksowano te same 500 000 dokumentów. Pola tekstowe `title` i `text` są indeksowane i przechowywane. Kategorie i data dodatkowo mają `docValues`, co ułatwia faceting i sortowanie. Wszystkie pola są pojedynczej wartości (`multiValued=false`). Automatyczne tworzenie pól schemaless wyłączono.

Zapytania korzystają z eDisMax i pól `title^3 text`, zwracają do 10 dokumentów posortowanych według score, a osobny przykład prezentuje highlighting oraz facety.

Zapytanie Solr	Dopasowane dokumenty
Słowo `run`	14 492
Fraza `United States`	38 188
Warunek logiczny	3 984

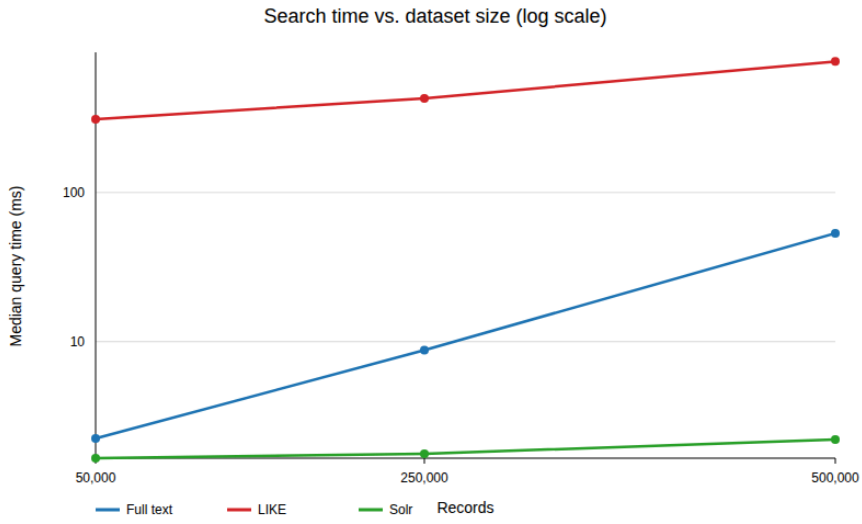
Ranking top-10 porównano po ID dokumentów. Wspólne wyniki: `run` — 0/10, fraza — 0/10, warunek logiczny — 0/10 (`csv/top10_comparison.csv`). Zapytania używają różnych mechanizmów analizy i rankingu: PostgreSQL `ts_rank_cd` z konfiguracją English, a Solr eDisMax z BM25 i polami `title^3 text`. W tym zbiorze skutkuje to różnymi pierwszymi dziesiątkami; nie oznacza to braku wyników zapytań, lecz brak zgodności ich rankingu.

Autouzupełnianie używa pola `title_prefix`, kopii tytułu analizowanej przez KeywordTokenizer, LowerCase i EdgeNGram (1–50 znaków). Dla prefiksu `foot` liczba wyników Solr wyniosła 113, zgodnie z liczbą tytułów zaczynających się od tego prefiksu w źródłowym CSV. SQL `lower(title) LIKE lower('foot%')` również zwrócił 113 wyników. Mediany pięciu pomiarów: SQL 194,380 ms, Solr 2,051 ms; szczegóły w `csv/autocomplete_results.csv`.

5. Skalowanie wyszukiwania

Każdy pomiar poprzedzono rozgrzewką, a czas zapytania mierzono pięć razy; raportowana jest mediana. Pomiar Solr wykorzystuje precyzyjny zegar klienta i obejmuje całe żądanie HTTP. Solr zwraca też pomocniczy, całkowity `QTime`, który dla szybkich zapytań wynosił 0 ms i nie nadawał się do wykresu.

Rekordy	SQL LIKE (ms)	SQL full text (ms)	Solr (ms, klient)
50 000	310,127	2,240	1,652
250 000	428,139	8,765	1,767
500 000	757,659	53,311	2,203

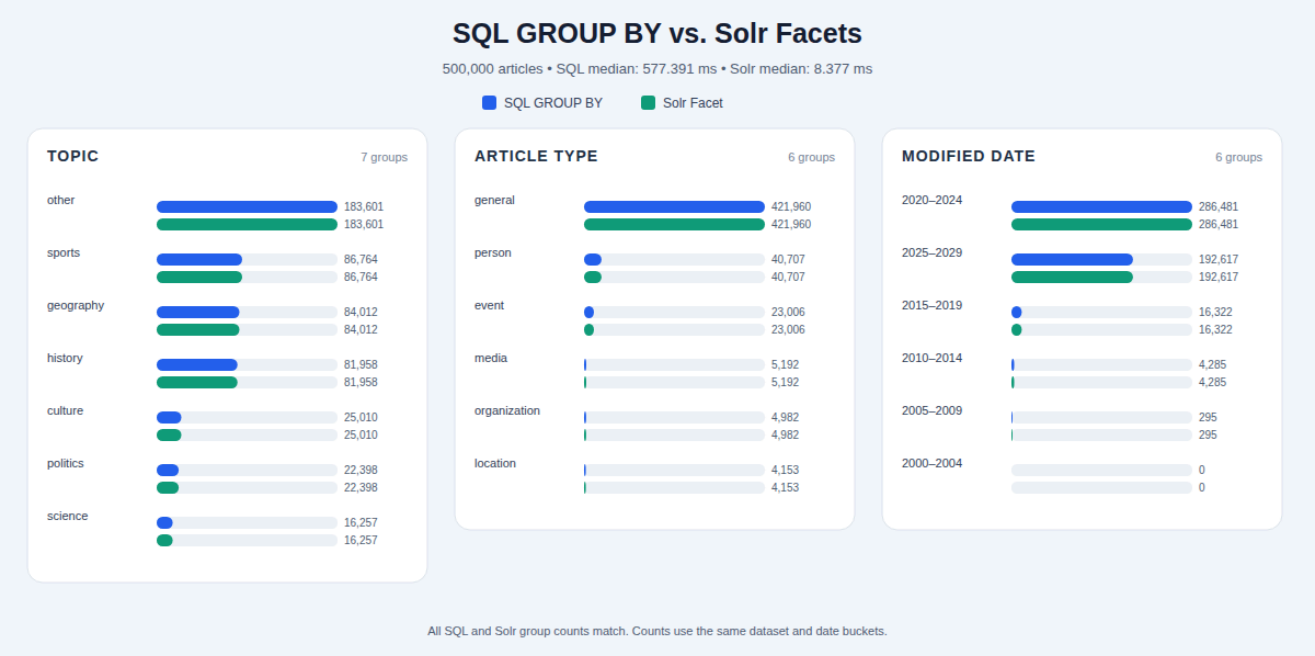


> **Interpretacja:** LIKE jest najwolniejszy w tym teście. Czasy SQL full-text i Solr także rosną wraz ze zbiorem, lecz dla Solra widoczny przyrost jest mniejszy. Porównanie nie jest w pełni równoważne: zapytania SQL i Solr używają innych analizatorów tekstu i zwracają inną liczbę dopasowań. Ponadto SQL mierzy czas wewnątrz serwera, a Solr po stronie klienta wraz z HTTP. Wnioski należy więc traktować jako pomiar tego wdrożenia, a nie uniwersalny ranking produktów.

6. GROUP BY kontra facety

W obu systemach pogrupowano 500 000 dokumentów po `topic`, `article_type` oraz dacie modyfikacji w zakresach pięcioletnich (2000–2029). Wyniki liczności zgadzają się dla wszystkich 19 grup.

Metoda	Mediana	Wyniki pomiarów
SQL GROUP BY (trzy grupowania razem)	577,391 ms	603,291; 558,830; 577,391; 551,552; 578,078
Solr — dwa facety kategoryczne i jeden zakresowy	8,377 ms	8,488; 8,325; 8,688; 8,377; 8,094



Czas SQL jest mierzony wewnątrz PostgreSQL, a czas Solr obejmuje klienta HTTP. Zakresy operacji są zbliżone, lecz pomiary mają różny narzut. Szczegóły liczności znajdują się w `csv/facet_comparison.csv`.

7. Rozszerzenia i kompletność wymagań

Rozszerzenie	Wynik
Autouzupełnianie (poziom 4.0)	Edge-ngram; dla `foot` 113 wyników w obu systemach. Mediany: SQL 194,380 ms; Solr 2,051 ms.
Głębokie stronicowanie: `start/rows` kontra `cursorMark` (poziom 5.0)	Offset 100 000, po 1 000 dokumentów; 5 rozgrzanych powtórzeń. `start/rows`: 376,914 ms / 1 żądanie; `cursorMark`: 1 362,369 ms / 101 żądań. Wszystkie 1 000 ID strony były identyczne. Dane w `csv/pagination_results.csv`.
Aktualizacje SQL → Solr (poziom 5.0)	Test zaliczony: INSERT 4, UPDATE 2 i DELETE 2 rekordów; po każdej operacji pola SQL i Solr były zgodne. Rekordy testowe usunięto.

> **Ważne:** pełne przejście cursorMark sprawdzono na 500 000 dokumentów (501 stron; `csv/cursor_mark_results.csv`). Dla aktualizacji automatycznych uruchom `python3 solr/sync_outbox.py --watch`; dostarczenie do Solr jest asynchroniczne.

8. Brakujące pomiary i ograniczenia

- Import Solr do istniejącego core'u po dodaniu edge-ngram (500 000 dokumentów) zajął 83,024 s. To reindeksacja istniejącego core'u, nie pomiar pierwszego, pustego importu.
- Czas pierwszego importu do PostgreSQL nie został zachowany. `database/measure_load.sh` mierzy import i budowę GIN na pustej tabeli głównej albo — jeśli jest już zapełniona — na osobnej tabeli roboczej, nie naruszając istniejących danych.

- Użyto jednego lokalnego środowiska; brak odtworzeń na innym sprzęcie i kontroli obciążenia systemu.
- Zapytania wyszukujące SQL i Solr mają różne analizatory, dlatego ich liczby trafień różnią się.
- Wartości niezmierzonych etapów nie zostały zastąpione szacunkami.

9. Pliki projektu i odtwarzanie

Instrukcje uruchomienia znajdują się w `README.md`. SQL znajduje się w `database/`, konfiguracja i narzędzia Solr w `solr/`, dane i walidator w `data-set/`, wyniki CSV w `csv/`, a wykresy SVG w `svg/`.

Przed oddaniem należy odtworzyć brakujące pomiary PostgreSQL. Test synchronizacji wielu rekordów został wykonany pomyślnie. Archiwum ZIP powinno zawierać kod, README i sprawozdanie, bez samego zbioru danych.